

APACHE DORIS 3.0.8

学习笔记

第五章：物化视图与查询加速

主线：把查询时的重复计算转移到写入或刷新阶段，再由 Nereids 在正确性和成本约束下复用物化结果。

配套编号：notes-05

基准版本：Apache Doris 3.0.8

整理日期：2026-08

目录

目录	2
本章学习目标	4
先建立成本意识	4
5.1 为什么需要物化视图	4
重复计算问题	4
物化的三个组成部分	5
与其他对象的边界	5
为什么会快	5
5.2 同步物化视图与异步物化视图	5
设计权衡	6
Unique Key 边界	6
5.3 同步物化视图的底层结构	6
从表到物理索引	6
预聚合示例	7
创建与历史回填	7
后续写入与强一致	7
5.4 同步物化视图如何被透明命中	7
计划结构而非文本相等	7
谓词包含方向	8
聚合方向	8
EXPLAIN 证据	8
5.5 异步物化视图的生命周期	8
DDL 的五个问题	8
Job 与 Task	9
刷新触发取舍	9
5.6 分区增量刷新原理	9
1:1 与多对一映射	9
多表复杂性	10
可增量的关键条件	10
AUTO 不等于保证增量	10
粒度取舍	10

5.7 透明改写与 SPJG	10
SPJG 结构	10
综合改写	11
主要改写能力	11
正确性优先	11
5.8 数据新鲜度与一致性	11
实际延迟组成	11
整体与分区状态	11
grace_period	11
设计原则	12
5.9 物化视图设计与资源治理	12
净收益模型	12
粒度与覆盖	12
分区与跟踪表	12
刷新与资源	12
上线监控	13
5.10 综合案例与命中失败诊断	13
两层加速架构	13
异步 MV 设计	13
标准故障路径	13
诊断命令	14
四类问题	14
第五章复习题	14
自测方法	15
术语速查	15
参考资料	15

本章学习目标

第四章研究一条 SQL 怎样执行得更好；第五章更进一步，研究怎样提前保存可复用的计算结果，使查询少扫描、少 Join、少聚合。学完后应能设计、验证并维护 Doris 3.0.8 的同步与异步物化视图。

问题域	应能回答	核心对象
定位	物化视图与 View、缓存、索引有什么区别？	预计算、物理存储、改写
同步 MV	为何它更像基础表内的 Rollup Index？	Base Index、Rollup、同事务
异步 MV	首次构建、刷新、任务和状态怎样协作？	BUILD、REFRESH、Job、Task
增量刷新	Doris 怎样找出需要刷新的分区？	映射、版本、失效分区
透明改写	Nereids 怎样证明物化结果可回答查询？	SPJG、补偿、Rollup、CBO
治理诊断	如何权衡新鲜度、收益、资源并定位不命中？	grace、Workload Group、EXPLAIN

第五章因果主线



先建立成本意识

$MV \approx \times$
 $MV \approx + \times +$

物化视图的价值不是让某条 SQL 看起来更高级，而是降低整个系统在一个时间窗口内的总计算成本。

5.1 为什么需要物化视图

物化视图把一段稳定、高频、昂贵的关系计算提前执行并保存，再由维护机制更新、由优化器复用。

重复计算问题

一个按天、地区汇总销售额的查询，输入可能是十亿级订单明细，输出只有数百行。如果每天执行数千次，即使单次执行已经优化，扫描、过滤和聚合仍在被重复支付。

没有物化时，每次查询都重走完整路径



物化的三个组成部分

组成	作用	缺失后的问题
定义 SQL	描述预计算什么	不知道保存什么语义
物理存储	保存可复用结果	仍需每次重新计算
维护刷新	跟随基础数据变化	结果逐渐过期
查询改写	让业务 SQL 自动复用结果	只能人工改查结果表

与其他对象的边界

对象	保存数据	查询是否重算	主要价值
普通 View	否	是	封装 SQL 与接口
物化视图	是	可复用预计算	跨查询复用关系结果
查询缓存	是	命中同类结果时否	复用最终结果
索引	保存辅助结构	仍执行剩余计算	定位或跳过数据
手工汇总表	是	通常否	业务自行维护计算结果

为什么会快

- 扫描更少的行和列，降低磁盘读取与解压。
- 提前完成 Join，减少在线网络 Shuffle 与 Hash 表。
- 提前完成聚合或复杂表达式，降低 CPU 与内存。
- 物化结果可使用独立分区、分桶、排序和索引布局。

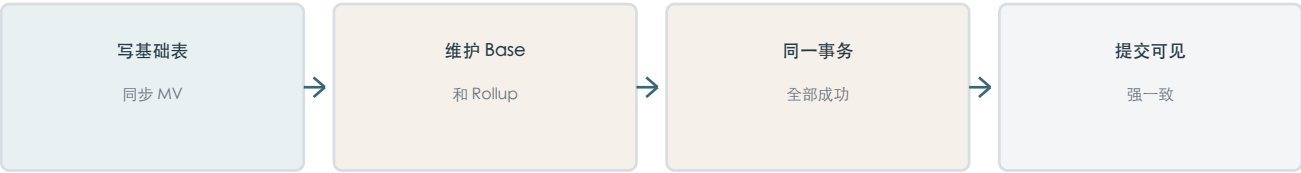
它没有让硬件变快，而是改变了查询必须完成的工作量。简单查询、小表或几乎不压缩数据的物化结果，可能无法覆盖刷新和存储成本。

5.2 同步物化视图与异步物化视图

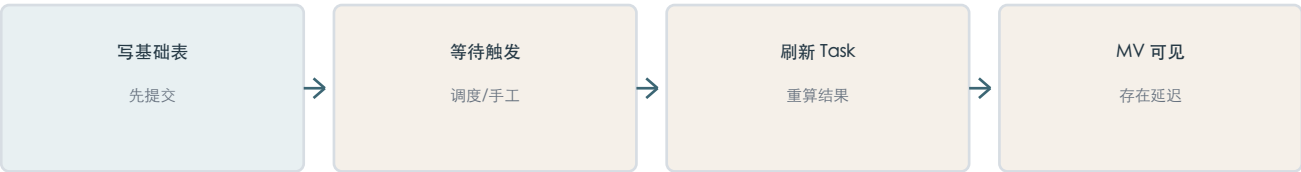
同步与异步描述基础表变化后物化结果何时维护，不描述 CREATE 语句返回得快不快。

维度	同步物化视图	异步物化视图
一致性	与基础表强一致	最终一致
维护时机	基础表写入时同步维护	由独立刷新任务维护
SQL 范围	主要为单表	支持复杂多表 SQL
直接查询	不支持，依赖透明改写	支持直接查询和透明改写
物理身份	基础表下的 Rollup Index	独立内部结果表
典型场景	实时单表聚合、排序重排	多表报表、建模、湖仓加速
主要代价	写放大	刷新资源与数据延迟

同步维护链路



异步维护链路



设计权衡

同步 MV 用额外写入成本换取实时一致与读取加速；异步 MV 接受可控延迟，换取更复杂的跨表预计算和与写入链路解耦。实时数仓通常组合使用，而不是二选一。

Unique Key 边界

Doris 3.0.8 中，同步物化视图在 Unique Key 模型上主要用于调整列顺序，不支持通过它进行聚合。需要更新语义的事实表应先明确数据模型，再决定用同步聚合、异步 MV 或直接查询。

5.3 同步物化视图的底层结构

同步物化视图不是独立业务表，而是同一逻辑表、同一分区拓扑下的一套额外 Materialized Index。

从表到物理索引

Table
Partition
Base Materialized Index
Tablet -> Rowset -> Segment -> Column/Page
Rollup Materialized Index (MV)
Tablet -> Rowset -> Segment -> Column/Page

特征	Base Index	同步 MV / Rollup Index
数据内容	表定义的基础数据	过滤、重排或预聚合结果
排序结构	基础表排序键	可有不同列顺序
物理文件	独立 Tablet/Rowset/Segment	也有独立物理文件
生命周期	逻辑表默认存在	依附基础表创建和删除
读取方式	优化器可选	优化器透明选择

预聚合示例

Base Index: 2026-08-01, order=1001, , 100
 2026-08-01, order=1002, , 200
 2026-08-01, order=1003, , 80
 Rollup MV : 2026-08-01, , 300
 2026-08-01, , 80

当 Base Index 每天数亿行而 Rollup 只有几十个分组时，扫描、解压和聚合成本会产生数量级差异。

创建与历史回填

同步 MV 的创建构建是后台任务



```
SHOW ALTER TABLE MATERIALIZED VIEW FROM your_database;
```

后续写入与强一致

构建完成后，每个导入事务同时生成 Base Index 和所有同步 MV 的 Rowset，相关索引全部成功后事务才提交。强一致来自同一提交边界，不是刷新线程跑得足够快。

$\approx \text{Base} + \text{MV1} + \text{MV2} + \dots$

同步 MV 越多，写放大、存储和 Compaction 成本越高，因此必须用查询收益证明每一套 Rollup 的价值。

5.4 同步物化视图如何被透明命中

命中分三关：对象可用、语义可改写、成本值得选择。能回答查询不等于最终一定被选中。

同步 MV 选择路径



计划结构而非文本相等

Doris 3.0 默认基于 Nereids 计划结构进行同步 MV 改写。别名、条件书写顺序或等价表达方式不同，不必然阻止命中；关键是标准化后的关系运算能否从物化结果推导。

谓词包含方向

MV 范围	查询范围	结果
status='PAID'	status='PAID' AND region='华东'	可在 MV 上补充过滤
status='PAID'	没有 status 条件	MV 缺少其他状态，不能单独回答
全部状态	status='PAID'	可继续过滤，但可能成本较高

聚合方向

MV 粒度为 dt+region，查询只按 dt 汇总时，可以再次 SUM；若 MV 只剩 dt，查询要求 dt+region，则被丢掉的 region 无法恢复。Rollup 只能由细到粗。

查询目标	MV 保存	再次聚合
SUM	SUM	SUM(mv_sum)
COUNT	COUNT	SUM(mv_count)
AVG	SUM + COUNT	总 SUM / 总 COUNT
精确去重	Bitmap 中间状态	BITMAP_UNION_COUNT
近似去重	HLL 中间状态	HLL 再合并

EXPLAIN 证据

```
EXPLAIN SELECT ... FROM base_table ...;
```

TABLE: db.base_table(mv_name)

PREAGGREGATION: ON

MaterializedViewRewriteSuccessAndChose: mv_name chose

SuccessButNotChose 表示语义成功但 CBO 认为其他方案更便宜；RewriteFail 表示状态、列、谓词、聚合或结构无法证明等价。

5.5 异步物化视图的生命周期

异步 MV 是由 Doris 管理的内部结果表：有独立存储和刷新 Job，可直接查询，也可被透明改写。

DDL 的五个问题

```
CREATE MATERIALIZED VIEW mv_sales
BUILD IMMEDIATE
REFRESH AUTO
ON SCHEDULE EVERY 1 HOUR
PARTITION BY (dt)
DISTRIBUTED BY HASH(region) BUCKETS 8
PROPERTIES ("workload_group"="mv_refresh_group")
AS SELECT ...;
```

部分	控制的问题	常见选择
BUILD	第一次何时构建	IMMEDIATE / DEFERRED
REFRESH	刷新哪些范围	AUTO / COMPLETE
ON	谁触发刷新	MANUAL / SCHEDULE / COMMIT

部分	控制的问题	常见选择
PARTITION BY	最小刷新与失效单元	跟随基础列或 date_trunc
DISTRIBUTED BY	数据怎样分布到 BE	HASH / RANDOM

Job 与 Task

一个 Job 对应多次历史 Task



刷新触发取舍

方式	适合	主要风险
ON MANUAL	上游批次完成后由调度触发	漏触发或依赖编排
ON SCHEDULE	稳定小时级、天级 SLA	排队与执行时间增加实际延迟
ON COMMIT	低频变化或上层 MV	高频写入产生刷新风暴

直接查询 MV 会读取当前已有结果；透明改写还要检查状态、新鲜度和查询等价性。刷新失败不等于对象消失，但可能使其无法安全改写。

5.6 分区增量刷新原理

增量刷新依赖基础分区到 MV 分区的映射和版本比较，不依赖扫描数据猜测更新时间。

1:1 与多对一映射

基础表	MV	变化后的刷新范围
按天分区	按天分区	某天变化只重刷对应日
按天分区	按月分区	某天变化需重刷对应整月

REFRESH AUTO 的核心流程



多表复杂性

若 MV 跟踪按天分区的订单事实表，订单某天变化可定位到对应日；非分区门店维表变化可能影响多个历史分区，Doris 无法只凭表版本确定影响日期，因而可能使大量甚至全部 MV 分区失效。

可增量的关键条件

- 至少一张基础表使用 Range 或 List 分区。
- MV 只有一个跟踪分区字段，且能追溯到基础表分区列。
- 该字段出现在 SELECT 中；有 GROUP BY 时满足分组要求。
- 可直接跟随分区列，或用支持的 date_trunc 做分区上卷。
- 不能依赖外连接 NULL 生成侧中无法可靠追踪的分区字段。

AUTO 不等于保证增量

如果分区映射无法建立，或非跟踪表变化使影响范围无法局部化，AUTO 可能退化为全量刷新。COMPLETE 总是强制刷新全部分区；PARTITIONS 则强制刷新指定分区。

```
REFRESH MATERIALIZED VIEW mv_sales AUTO;
REFRESH MATERIALIZED VIEW mv_sales PARTITIONS (p20260818);
REFRESH MATERIALIZED VIEW mv_sales COMPLETE;
```

粒度取舍

分区越细，刷新定位越精确，但分区和 Tablet 元数据越多；分区越粗，元数据更少，但一天变化可能重算整月。应以数据封闭周期、单分区刷新成本和常见查询范围共同决定。

5.7 透明改写与 SPJG

透明改写是在关系代数等价前提下，将扫描基础表的逻辑计划替换为扫描 MV 并执行必要补偿。

SPJG 结构

字母	算子	含义
S	Select	过滤满足条件的行
P	Project	选择列和计算表达式
J	Join	连接多张关系
G	Group By	分组合并聚合状态

异步 MV 透明改写的证明路径



综合改写

```
MV : + + , Join
: + , dt >= '2026-08-01'

: MV -> -> SUM(mv_sales)
: Join
```

主要改写能力

能力	核心条件	改写动作
谓词补偿	查询范围是 MV 范围的子集	在 MV 上增加过滤
Join 改写	关系图和 Join 语义可映射	复用已连接结果
聚合 Rollup	MV 维度包含查询维度	再次合并中间状态
分区补偿	有效与失效范围可分离	MV UNION ALL 基础表
嵌套改写	上层 MV 可复用下层结果	多级预计算

正确性优先

优化器必须证明表关系、Join/NULL 语义、过滤范围、列覆盖、聚合粒度、分区完整性均正确。任何一环无法证明都应放弃改写：宁可慢，也不能返回错误结果。

5.8 数据新鲜度与一致性

刷新周期、实际数据延迟和透明改写可用性是三个不同概念。grace_period 只控制旧数据能否被改写使用。

实际延迟组成

= + + +

每 10 分钟调度一次，并不等于数据最多延迟 10 分钟。SLA 应包含刷新耗时和资源拥塞。

整体与分区状态

```
SELECT * FROM mv_infos("database"="rt_dw") WHERE Name="mv_sales";
SHOW PARTITIONS FROM mv_sales;
```

信号	含义	下一步
RefreshState=FAIL	最近一次刷新失败	查询 tasks 的 ErrorMsg
SyncWithBaseTables=0	至少部分数据不同步	定位失效分区
State=SCHEMA_CHANGE	基础 Schema 影响 MV	查看 SchemaChangeDetail
分区 Sync=1	该分区仍与基础同步	查询只涉及它时可正常改写

grace_period

grace_period=0 表示透明改写要求同步；grace_period=600 表示业务明确接受约 10 分钟延迟。它不会触发刷新，也不会保证直接查询 MV 自动补齐数据。

分区可用性决策



设计原则

先确定业务可以接受的延迟，再设计刷新周期与资源，最后设置 `grace_period`。不要用很大的容忍窗口掩盖长期失败的刷新任务。

5.9 物化视图设计与资源治理

从查询族与总成本出发：用一套可维护的预计算覆盖多个高频查询，而不是每张报表创建一个 MV。

净收益模型

$$\approx \begin{matrix} \times \\ - \\ - \end{matrix} \times$$

粒度与覆盖

选择	收益	代价
较粗粒度	行数少、扫描快、存储少	不能回答更细维度
较细粒度	覆盖更多查询、可再 Rollup	刷新和扫描成本增加
更多列	过滤和投影覆盖更广	列宽、I/O、存储增加
更多 MV	单查询更容易精确命中	刷新、运维、CBO 搜索增加

分区与跟踪表

- 优先让大、按时间新增、分区清晰的事实表作为跟踪表。
- 事实表变化应尽量只使局部分区失效；其他维表最好低频变化。
- 分区粒度同时考虑刷新最小单元、分区数量和查询时间范围。

刷新与资源

场景	更合适的方式	注意
分钟级强实时	同步 MV 或直接查询	异步刷新通常难稳定承诺
上游批次	ON MANUAL	批次成功后显式触发
小时/天级报表	ON SCHEDULE	错峰并纳入执行耗时
低频变化	ON COMMIT	防止高频触发刷新风暴

用 workload_group 隔离刷新 CPU 和内存；refresh_partition_num 控制单批处理分区数。配额过大影响在线查询，过小可能导致 Hash Join 或聚合刷新失败。

上线监控

- 查询：命中率、延迟、扫描行数和规划耗时。
- 刷新：成功率、排队、执行时间、失败分区和数据延迟。
- 资源：CPU、内存、网络、I/O、存储和 Tablet 数量。

物化视图本质上是增强型 ETL，需要持续维护、容量规划和下线长期不命中的对象。

5.10 综合案例与命中失败诊断

实时单表指标用同步 MV；允许延迟的复杂多表经营分析用异步 MV。两条链路共同服务实时数仓。

两层加速架构

同一事实数据的实时与经营分析路径



异步 MV 设计

```

BUILD IMMEDIATE
REFRESH AUTO ON SCHEDULE EVERY 15 MINUTE
PARTITION BY (event_date)
DISTRIBUTED BY HASH(region_id) BUCKETS 16
PROPERTIES (
  "grace_period"="1200",
  "workload_group"="mv_refresh_group",
  "refresh_partition_num"="1"
)
AS SELECT event_date, event_hour, region_id, category_id, store_id,
        SUM(pay_amount), COUNT(*)
FROM fact JOIN dimensions ...
GROUP BY event_date, event_hour, region_id, category_id, store_id;
  
```

该粒度可以向上回答按天、地区、品类等报表，但不能恢复用户、订单或 SKU 级信息。它以覆盖高频经营查询为目标，而非取代所有明细分析。

标准故障路径

1. EXPLAIN 确认是 Chose、NotChose 还是 RewriteFail。
2. MV_INFOS 查看 State、RefreshState 与整体同步状态。
3. SHOW PARTITIONS 定位具体失效分区和 UnsyncTables。
4. TASKS 查看 ErrorMessage、耗时、需要与已完成分区。
5. 区分状态、语义、成本和资源四类问题。
6. 修复后补刷目标分区，再次 EXPLAIN 与 Profile。

诊断命令

```
SHOW CREATE MATERIALIZED VIEW mv_name;
SELECT * FROM mv_infos("database"="db") WHERE Name="mv_name";
SHOW PARTITIONS FROM mv_name;
SELECT * FROM jobs("type"="mv") WHERE MvName="mv_name";
SELECT * FROM tasks("type"="mv") WHERE MvName="mv_name";
EXPLAIN SELECT ...;
EXPLAIN MEMO PLAN SELECT ...;
```

四类问题

类型	典型证据	方向
状态	Refresh FAIL、Sync=0	修复任务并补刷
语义	RewriteFail、缺列/谓词/粒度不兼容	重设 MV 或改查询
成本	SuccessButNotChose	统计、扫描量、布局
资源	Memory limit、排队、倾斜	Workload Group、分桶、批次

最终最重要的诊断习惯：不要把‘未命中’统称为刷新问题。对象健康、语义等价和成本选择分别需要不同证据。

第五章复习题

1. 普通 View、查询缓存和物化视图的核心区别是什么？
2. 为什么物化视图快的本质是少做工作，而不是 CPU 变快？
3. 同步物化视图为什么更接近 Rollup Materialized Index？
4. 同步物化视图如何保证与基础表强一致？
5. 为什么创建同步物化视图是后台任务，但仍然称为同步？
6. 同步物化视图语义匹配成功后，为什么 CBO 仍可能选择 Base Index？
7. BUILD、REFRESH 和 ON 分别控制什么？
8. REFRESH AUTO 为什么不保证一定执行分区增量刷新？
9. Doris 怎样通过分区映射和版本判断需要刷新的 MV 分区？
10. 多表 MV 中非分区维表变化，为什么可能使大量分区失效？
11. SPJG 分别表示什么？
12. 什么是谓词补偿？为什么查询范围比 MV 更大时通常不能直接改写？
13. MV 粒度为天、地区、用户，为什么可以回答天、地区查询？
14. 为什么不同分组的 AVG 不能直接再次求平均？
15. 分区补偿改写为什么可以使用 UNION ALL 而不产生重复数据？
16. grace_period 控制刷新周期还是透明改写容忍度？
17. SyncWithBaseTables=0 是否意味着整张分区 MV 完全不可使用？
18. 如何在细粒度、高复用与低刷新成本之间选择 MV 粒度？

19. 为什么大量高频刷新的 MV 可能拖慢在线查询？

20. 面对物化视图未命中，怎样区分状态、语义和成本问题？

自测方法

每题先给一句话结论，再补齐数据流、正确性边界、成本取舍和一个可观测证据。答案请使用独立的 notes-05-a。

术语速查

术语	一句话含义
Materialization	把关系计算结果物理保存以便复用
Base Index	逻辑表默认的基础 Materialized Index
Rollup Index	基础表下额外的物理排列或聚合索引
Async MV	独立刷新、最终一致的内部结果表
Job / Task	长期刷新规则与一次具体刷新执行
Partition Mapping	基础分区到 MV 分区的对应关系
SPJG	Select、Project、Join、Group By 结构
Predicate Compensation	在 MV 上补充查询更严格的过滤
grace_period	透明改写允许的数据延迟窗口
SyncWithBaseTables	MV 整体或分区是否与基础表同步

参考资料

以下链接用于核对 Doris 3.0.8 的物化视图语法、刷新、状态与改写机制。

- [1] Doris 3.0 Materialized View Overview
- [2] Doris 3.0 Create Sync Materialized View
- [3] Sync Materialized View
- [4] Doris 3.0 Async MV Management
- [5] Create Async Materialized View
- [6] Doris 3.0 Refresh Materialized View
- [7] Doris 3.0 Async MV FAQ
- [8] Transparent Rewrite with Async MV
- [9] Doris 3.0 MV_INFOS
- [10] Doris 3.0 TASKS
- [11] Doris 3.0 JOBS
- [12] Doris 3.0.0 Release Notes
- [13] Release 3.0.8